

1. InputStream and OutputStream

InputStream and **OutputStream** are abstract classes that form the foundation for byte-oriented input and output operations in Java.

InputStream

- **Purpose:** Used to read bytes from a source (like a file, network socket, etc.).
- **Key Methods:**
 - `public abstract int read() throws IOException`: Reads the next byte of data from the input stream. Returns -1 if the end of the stream is reached.
 - `public int available() throws IOException`: Returns an estimate of the number of bytes that can be read from the input stream without blocking.
 - `public void close() throws IOException`: Closes the input stream and releases any associated resources.

Example:

```
java
VerifyOpen In EditorRunCopy code
1import java.io.FileInputStream;
2import java.io.IOException;
3
4public class InputStreamExample {
5    public static void main(String[] args) {
6        FileInputStream inputStream = null;
7        try {
8            inputStream = new FileInputStream("example.txt");
9            int data;
10           while ((data = inputStream.read()) != -1) {
11               System.out.print((char) data); // Print each byte as a
character
12           }
13       } catch (IOException e) {
14           e.printStackTrace();
15       } finally {
16           try {
17               if (inputStream != null) {
18                   inputStream.close();
19               }
20           } catch (IOException e) {
21               e.printStackTrace();
22           }
23       }
24   }
25}
```

OutputStream

- **Purpose:** Used to write bytes to a destination (like a file, network socket, etc.).
- **Key Methods:**
 - `public void write(int b) throws IOException`: Writes the specified byte to the output stream.
 - `public void write(byte[] b) throws IOException`: Writes an array of bytes to the output stream.

- `public void flush()` throws `IOException`: Flushes the output stream, ensuring that any buffered output bytes are written out.
- `public void close()` throws `IOException`: Closes the output stream and releases any associated resources.

Example:

```

java
VerifyOpen In EditorRunCopy code
1import java.io.FileOutputStream;
2import java.io.IOException;
3
4public class OutputStreamExample {
5    public static void main(String[] args) {
6        FileOutputStream outputStream = null;
7        try {
8            outputStream = new FileOutputStream("output.txt");
9            String data = "Hello, World!";
10           outputStream.write(data.getBytes()); // Write string as bytes
11        } catch (IOException e) {
12            e.printStackTrace();
13        } finally {
14            try {
15                if (outputStream != null) {
16                    outputStream.close();
17                }
18            } catch (IOException e) {
19                e.printStackTrace();
20            }
21        }
22    }
23}

```

2. Reader and Writer

Reader and **Writer** are abstract classes designed for character-oriented input and output operations.

Reader

- **Purpose:** Used to read characters from a source (like a file, string, etc.).
- **Key Methods:**
 - `public abstract int read()` throws `IOException`: Reads a single character from the input stream.
 - `public int read(char[] cbuf)` throws `IOException`: Reads characters into a portion of an array.
 - `public void close()` throws `IOException`: Closes the reader and releases any associated resources.

Example:

```

java
VerifyOpen In EditorRunCopy code
1import java.io.FileReader;
2import java.io.IOException;
3
4public class ReaderExample {
5    public static void main(String[] args) {
6        FileReader reader = null;

```

```

7      try {
8          reader = new FileReader("example.txt");
9          int data;
10         while ((data = reader.read()) != -1) {
11             System.out.print((char) data); // Print each character
12         }
13     } catch (IOException e) {
14         e.printStackTrace();
15     } finally {
16         try {
17             if (reader != null) {
18                 reader.close();
19             }
20         } catch (IOException e) {
21             e.printStackTrace();
22         }
23     }
24 }
25}

```

Writer

- **Purpose:** Used to write characters to a destination (like a file, string, etc.).
- **Key Methods:**
 - `public void write(int c) throws IOException:` Writes a single character to the output stream.
 - `public void write(char[] cbuf) throws IOException:` Writes an array of characters to the output stream.
 - `public void flush() throws IOException:` Flushes the output stream.
 - `public void close() throws IOException:` Closes the writer and releases any associated resources.

Example:

```

java
VerifyOpen In EditorRunCopy code
1import java.io.FileWriter;
2import java.io.IOException;
3
4public class WriterExample {
5    public static void main(String[] args) {
6        FileWriter writer = null;
7        try {
8            writer = new FileWriter("output.txt");
9            writer.write("Hello, World!"); // Write string to file
10        } catch (IOException e) {
11            e.printStackTrace();
12        } finally {
13            try {
14                if (writer != null) {
15                    writer.close();
16                }
17            } catch (IOException e) {
18                e.printStackTrace();
19            }
20        }
21    }
22}

```

3. Summary of Differences

- **InputStream/OutputStream:** Handle byte data (e.g., images, binary files).
- **Reader/Writer:** Handle character data (e.g., text files).
- **Bridging Classes:** `InputStreamReader` and `OutputStreamWriter` can convert between byte streams and character streams using a specified character set.

Conclusion

The `java.io` package provides a comprehensive set of classes for handling input and output operations in Java. Understanding the differences between byte streams and character streams is crucial for effective file handling and data processing in Java applications.

Java NIO Package: Detailed Explanation with Examples

Java NIO (New Input/Output) is a powerful library introduced in Java 1.4 that provides an alternative to the traditional Java I/O API. It is designed for high-performance I/O operations and supports non-blocking I/O, which allows for more scalable applications. Below is a detailed overview of the key components of the Java NIO package, along with examples.

Key Components of Java NIO

1. Buffers

- Buffers are containers for data of a specific primitive type (e.g., byte, char, int).
- They provide a way to read and write data in a more efficient manner compared to traditional streams.

Example: Using ByteBuffer

```
java
VerifyOpen In EditorRunCopy code
1import java.nio.ByteBuffer;
2
3public class ByteBufferExample {
4    public static void main(String[] args) {
5        // Allocate a ByteBuffer of size 10
6        ByteBuffer buffer = ByteBuffer.allocate(10);
7
8        // Write data to the buffer
9        buffer.put((byte) 1);
10       buffer.put((byte) 2);
11       buffer.put((byte) 3);
12
13       // Flip the buffer for reading
14       buffer.flip();
15
16       // Read data from the buffer
17       while (buffer.hasRemaining()) {
18           System.out.println(buffer.get());
19       }
20   }
21}
```

2. Channels

- Channels represent connections to entities capable of performing I/O operations, such as files and sockets.
- They can be blocking or non-blocking.

Example: Using FileChannel

```
java
VerifyOpen In EditorRunCopy code
1import java.io.FileInputStream;
2import java.io.FileOutputStream;
3import java.nio.ByteBuffer;
4import java.nio.channels.FileChannel;
5
6public class FileChannelExample {
7    public static void main(String[] args) {
8        try (FileInputStream fis = new FileInputStream("input.txt");
9             FileOutputStream fos = new FileOutputStream("output.txt");
10             FileChannel inputChannel = fis.getChannel();
11             FileChannel outputChannel = fos.getChannel()) {
12
13            ByteBuffer buffer = ByteBuffer.allocate(1024);
14
15            // Read data from input file
16            int bytesRead = inputChannel.read(buffer);
17            while (bytesRead != -1) {
18                buffer.flip(); // Switch to read mode
19                outputChannel.write(buffer); // Write to output file
20                buffer.clear(); // Clear the buffer for the next read
21                bytesRead = inputChannel.read(buffer);
22            }
23        } catch (Exception e) {
24            e.printStackTrace();
25        }
26    }
27}
```

3. Selectors

- Selectors allow a single thread to monitor multiple channels for events (e.g., connection opened, data arrived).
- This is useful for implementing non-blocking I/O.

Example: Using Selector with SocketChannel

```
java
VerifyOpen In EditorRunCopy code
1import java.io.IOException;
2import java.net.InetSocketAddress;
3import java.nio.ByteBuffer;
4import java.nio.channels.Selector;
5import java.nio.channels.SelectionKey;
6import java.nio.channels.ServerSocketChannel;
7import java.nio.channels.SocketChannel;
8import java.util.Iterator;
9
10public class SelectorExample {
11    public static void main(String[] args) throws IOException {
12        Selector selector = Selector.open();
13        ServerSocketChannel serverChannel = ServerSocketChannel.open();
14        serverChannel.bind(new InetSocketAddress(8080));
```

```

15     serverChannel.configureBlocking(false);
16     serverChannel.register(selector, SelectionKey.OP_ACCEPT);
17
18     while (true) {
19         selector.select(); // Wait for events
20         Iterator<SelectionKey> keys =
selector.selectedKeys().iterator();
21         while (keys.hasNext()) {
22             SelectionKey key = keys.next();
23             if (key.isAcceptable()) {
24                 SocketChannel clientChannel =
serverChannel.accept();
25                 clientChannel.configureBlocking(false);
26                 clientChannel.register(selector,
SelectionKey.OP_READ);
27             } else if (key.isReadable()) {
28                 SocketChannel clientChannel = (SocketChannel)
key.channel();
29                 ByteBuffer buffer = ByteBuffer.allocate(256);
30                 int bytesRead = clientChannel.read(buffer);
31                 if (bytesRead == -1) {
32                     clientChannel.close();
33                 } else {
34                     buffer.flip();
35                     System.out.println("Received: " + new
String(buffer.array(), 0, bytesRead));
36                 }
37             }
38             keys.remove();
39         }
40     }
41 }
42}

```

4. Charsets

- Java NIO provides support for character encoding and decoding through the `java.nio.charset` package.
- This allows for the conversion between bytes and characters.

Example: Using Charset

```

java
VerifyOpen In EditorRunCopy code
1import java.nio.charset.Charset;
2
3public class CharsetExample {
4    public static void main(String[] args) {
5        String original = "Hello, World!";
6        Charset charset = Charset.forName("UTF-8");
7
8        // Encode the string into bytes
9        byte[] bytes = original.getBytes(charset);
10
11        // Decode the bytes back into a string
12        String decoded = new String(bytes, charset);
13        System.out.println(decoded);
14    }
15}

```

Conclusion

Java NIO provides a robust framework for handling I/O operations in a more efficient and scalable manner compared to traditional Java I/O. By utilizing buffers, channels, selectors, and charsets, developers can create high-performance applications that can handle multiple I/O operations concurrently. The examples provided illustrate the basic usage of these components, and you can expand upon them for more complex applications.

Java Serialization and Deserialization: Detailed Explanation with Examples

Overview of Serialization and Deserialization

- **Serialization** is the process of converting an object's state into a byte stream, which can be saved to a file or transmitted over a network.
- **Deserialization** is the reverse process, where the byte stream is converted back into an object.

Key Concepts

- **Serializable Interface:** To make a class serializable, it must implement the `java.io.Serializable` interface, which is a marker interface (it does not contain any methods).
- **ObjectOutputStream:** Used for serialization, it provides the `writeObject()` method to serialize an object.
- **ObjectInputStream:** Used for deserialization, it provides the `readObject()` method to reconstruct the object from the byte stream.
- **serialVersionUID:** A unique identifier for each serializable class, used to ensure that a loaded class corresponds exactly to a serialized object.

Example 1: Basic Serialization and Deserialization

Here's a simple example demonstrating serialization and deserialization in Java.

```
java
VerifyOpen In EditorRunCopy code
1import java.io.*;
2
3// Class implementing Serializable
4class Demo implements Serializable {
5    public int a;
6    public String b;
7
8    // Constructor
9    public Demo(int a, String b) {
10        this.a = a;
11        this.b = b;
12    }
13}
14
15public class Test {
16    public static void main(String[] args) {
17        Demo object = new Demo(1, "geeksforgeeks");
18        String filename = "file.ser";
19
```

```

20 // Serialization
21 try {
22     FileOutputStream file = new FileOutputStream(filename);
23     ObjectOutputStream out = new ObjectOutputStream(file);
24     out.writeObject(object);
25     out.close();
26     file.close();
27     System.out.println("Object has been serialized");
28 } catch (IOException ex) {
29     System.out.println("IOException is caught");
30 }
31
32 // Deserialization
33 Demo object1 = null;
34 try {
35     FileInputStream file = new FileInputStream(filename);
36     ObjectInputStream in = new ObjectInputStream(file);
37     object1 = (Demo) in.readObject();
38     in.close();
39     file.close();
40     System.out.println("Object has been deserialized");
41     System.out.println("a = " + object1.a);
42     System.out.println("b = " + object1.b);
43 } catch (IOException ex) {
44     System.out.println("IOException is caught");
45 } catch (ClassNotFoundException ex) {
46     System.out.println("ClassNotFoundException is caught");
47 }
48 }
49}

```

Example 2: Serialization with Transient and Static Variables

In this example, we will see how transient and static variables behave during serialization.

```

java
VerifyOpen In EditorRunCopy code
1import java.io.*;
2
3class Emp implements Serializable {
4    private static final long serialVersionUID = 1L;
5    transient int a; // This will not be serialized
6    static int b; // This will not be serialized
7    String name;
8    int age;
9
10    // Constructor
11    public Emp(String name, int age, int a) {
12        this.name = name;
13        this.age = age;
14        this.a = a;
15    }
16}
17
18public class SerialExample {
19    public static void printData(Emp object) {
20        System.out.println("name = " + object.name);
21        System.out.println("age = " + object.age);
22        System.out.println("a = " + object.a);
23        System.out.println("b = " + Emp.b);
24    }
25
26    public static void main(String[] args) {

```



```

27     Emp object = new Emp("John", 30, 5);
28     String filename = "emp.ser";
29
30     // Serialization
31     try {
32         FileOutputStream file = new FileOutputStream(filename);
33         ObjectOutputStream out = new ObjectOutputStream(file);
34         out.writeObject(object);
35         out.close();
36         file.close();
37         System.out.println("Object has been serialized");
38         printData(object);
39     } catch (IOException ex) {
40         System.out.println("IOException is caught");
41     }
42
43     // Deserialization
44     object = null;
45     try {
46         FileInputStream file = new FileInputStream(filename);
47         ObjectInputStream in = new ObjectInputStream(file);
48         object = (Emp) in.readObject();
49         in.close();
50         file.close();
51         System.out.println("Object has been deserialized");
52         printData(object);
53     } catch (IOException ex) {
54         System.out.println("IOException is caught");
55     } catch (ClassNotFoundException ex) {
56         System.out.println("ClassNotFoundException is caught");
57     }
58 }
59}

```

Important Points to Remember

- **Transient Variables:** Variables marked as `transient` are not serialized. They will be initialized to their default values during deserialization.
- **Static Variables:** Static variables are not serialized as they belong to the class rather than any instance of the class.
- **serialVersionUID:** Always declare a `serialVersionUID` in your serializable classes to maintain version control during serialization and deserialization.

Conclusion

Serialization and deserialization are powerful features in Java that allow for object persistence and communication between different Java Virtual Machines (JVMs). By understanding how to implement these processes, you can effectively manage object states in your applications.

Java Shallow Copy vs Deep Copy: Detailed Explanation with Examples

In Java, copying objects can be done in two primary ways: **shallow copy** and **deep copy**. Understanding the differences between these two methods is crucial for effective memory management and object manipulation in Java.

1. Shallow Copy

- **Definition:** A shallow copy of an object creates a new object but does not create copies of the objects that the original object references. Instead, it copies the references to those objects. Therefore, both the original and the copied object share the same references to the nested objects.
- **Characteristics:**
 - Fast, as it does not allocate new memory for the referenced objects.
 - Changes made to the mutable objects in the copied object will reflect in the original object since they share the same references.
- **Example:**

```
java
VerifyOpen In EditorRunCopy code
1class ABC {
2    int x = 30;
3}
4
5public class ShallowCopyExample {
6    public static void main(String[] args) {
7        // Creating an object of the class ABC
8        ABC obj1 = new ABC();
9
10       // Creating a shallow copy
11       ABC obj2 = obj1; // obj2 references the same object as obj1
12
13       // Updating the value using obj2
14       obj2.x = 6;
15
16       // Printing the value of x using obj1
17       System.out.println("The value of x is: " + obj1.x); // Output: 6
18   }
19}
```

- **Output:**

```
VerifyOpen In EditorRunCopy code
1The value of x is: 6
```

- **Explanation:** In this example, `obj1` and `obj2` refer to the same instance of `ABC`. Changing `obj2.x` also changes `obj1.x` because they share the same memory location.
-

2. Deep Copy

- **Definition:** A deep copy creates a new object and recursively copies all objects referenced by the original object. This means that the new object and the original object do not share any references; they are completely independent.
- **Characteristics:**
 - Slower, as it involves creating new instances of all referenced objects.
 - Changes made to the copied object do not affect the original object.
- **Example:**

```
java
```

VerifyOpen In EditorRunCopy code

```
1class ABC {
2    int x = 30;
3}
4
5public class DeepCopyExample {
6    public static void main(String[] args) {
7        // Creating an object of the class ABC
8        ABC obj1 = new ABC();
9
10       // Creating a deep copy
11       ABC obj2 = new ABC(); // obj2 is a new instance
12
13       // Updating the value using obj2
14       obj2.x = 6;
15
16       // Printing the value of x using obj1
17       System.out.println("The value of x is: " + obj1.x); // Output: 30
18   }
19}
```

- **Output:**

VerifyOpen In EditorRunCopy code

```
1The value of x is: 30
```

- **Explanation:** Here, obj1 and obj2 are two distinct instances of ABC. Changing obj2.x does not affect obj1.x because they do not share the same memory location.

3. Differences Between Shallow Copy and Deep Copy

Feature	Shallow Copy	Deep Copy
Memory Allocation	No new memory for referenced objects	New memory allocated for all objects
Performance	Faster	Slower
Reference Sharing	Shares references	No shared references
Impact of Changes	Changes in one affect the other	Changes in one do not affect the other
Clone Method Support	Default clone() method supports	Requires overriding clone() method

4. Special Case: Strings and Primitive Types

- **Strings:** In Java, strings are immutable. When you copy a string, you are copying the reference, but any modification results in a new string object being created. Thus, copying strings does not fit neatly into shallow or deep copy categories.
- **Primitive Types:** For primitive data types (like int, boolean), copying simply copies the value. There is no concept of shallow or deep copy since they are not objects.

java

VerifyOpen In EditorRunCopy code

```
1public class DataTypeCopyExample {
2    public static void main(String[] args) {
```

```

3      int x = 9;
4      int y = x; // Copying value
5
6      System.out.println("The value of x & y are: " + x + ", " + y); //
Output: 9, 9
7
8      // Updating y
9      y = 10;
10     System.out.println("The value of x & y are: " + x + ", " + y); //
Output: 9, 10
11 }
12}

```

- **Output:**

VerifyOpen In EditorRunCopy code

```

1The value of x & y are: 9, 9
2The value of x & y are: 9, 10

```

Conclusion

Understanding the differences between shallow and deep copies is essential for effective object management in Java. Use shallow copies for efficiency when shared references are acceptable, and deep copies when you need complete independence between objects.

Final Answer

Java I/O and NIO Packages: Detailed Explanation

In this response, I will provide a comprehensive overview of the `java.io` and `java.nio` packages in Java, covering the `InputStream`, `OutputStream`, `Reader`, `Writer` interfaces, the `NIO` package, serialization and deserialization, as well as shallow and deep copies. Each section will include key points, subpoints, and suitable examples to enhance your understanding.

1. Java I/O Package

The `java.io` package provides classes and interfaces for system input and output through data streams, serialization, and the file system.

1.1 `InputStream` and `OutputStream`

`InputStream` and **`OutputStream`** are abstract classes for byte-oriented input and output.

`InputStream`

- **Purpose:** Reads bytes from a source.
- **Key Methods:**
 - `int read()`: Reads the next byte of data.
 - `int available()`: Returns the number of bytes that can be read.
 - `void close()`: Closes the stream.

Example:

```
java
VerifyOpen In EditorRunCopy code
1import java.io.FileInputStream;
2import java.io.IOException;
3
4public class InputStreamExample {
5    public static void main(String[] args) {
6        try (FileInputStream inputStream = new FileInputStream("example.txt"))
7        {
8            int data;
9            while ((data = inputStream.read()) != -1) {
10                System.out.print((char) data);
11            }
12        } catch (IOException e) {
13            e.printStackTrace();
14        }
15}
```

OutputStream

- **Purpose:** Writes bytes to a destination.
- **Key Methods:**
 - `void write(int b):` Writes a byte.
 - `void write(byte[] b):` Writes an array of bytes.
 - `void flush():` Flushes the stream.

Example:

```
java
VerifyOpen In EditorRunCopy code
1import java.io.FileOutputStream;
2import java.io.IOException;
3
4public class OutputStreamExample {
5    public static void main(String[] args) {
6        try (FileOutputStream outputStream = new
7FileOutputStream("output.txt")) {
8            String data = "Hello, World!";
9            outputStream.write(data.getBytes());
10        } catch (IOException e) {
11            e.printStackTrace();
12        }
13}
```

1.2 Reader and Writer

Reader and **Writer** are abstract classes for character-oriented input and output.

Reader

- **Purpose:** Reads characters from a source.
- **Key Methods:**
 - `int read():` Reads a single character.
 - `int read(char[] cbuf):` Reads characters into an array.
 - `void close():` Closes the reader.

Example:

```
java
VerifyOpen In EditorRunCopy code
1import java.io.FileReader;
2import java.io.IOException;
3
4public class ReaderExample {
5    public static void main(String[] args) {
6        try (FileReader reader = new FileReader("example.txt")) {
7            int data;
8            while ((data = reader.read()) != -1) {
9                System.out.print((char) data);
10            }
11        } catch (IOException e) {
12            e.printStackTrace();
13        }
14    }
15}
```

Writer

- **Purpose:** Writes characters to a destination.
- **Key Methods:**
 - `void write(int c):` Writes a single character.
 - `void write(char[] cbuf):` Writes an array of characters.
 - `void flush():` Flushes the writer.

Example:

```
java
VerifyOpen In EditorRunCopy code
1import java.io.FileWriter;
2import java.io.IOException;
3
4public class WriterExample {
5    public static void main(String[] args) {
6        try (FileWriter writer = new FileWriter("output.txt")) {
7            writer.write("Hello, World!");
8        } catch (IOException e) {
9            e.printStackTrace();
10        }
11    }
12}
```

2. Java NIO Package

Java NIO (New Input/Output) is designed for high-performance I/O operations and supports non-blocking I/O.

2.1 Key Components of Java NIO

1. **Buffers:** Containers for data of a specific type.
 - **Example:** Using `ByteBuffer`.

```
java
VerifyOpen In EditorRunCopy code
```

```

1import java.nio.ByteBuffer;
2
3public class ByteBufferExample {
4    public static void main(String[] args) {
5        ByteBuffer buffer = ByteBuffer.allocate(10);
6        buffer.put((byte) 1);
7        buffer.flip();
8        while (buffer.hasRemaining()) {
9            System.out.println(buffer.get());
10        }
11    }
12}

```

2. **Channels:** Represent connections to entities capable of performing I/O operations.

- **Example:** Using `FileChannel`.

```

java
VerifyOpen In EditorRunCopy code
1import java.io.FileInputStream;
2import java.io.FileOutputStream;
3import java.nio.ByteBuffer;
4import java.nio.channels.FileChannel;
5
6public class FileChannelExample {
7    public static void main(String[] args) {
8        try (FileInputStream fis = new FileInputStream("input.txt");
9             FileOutputStream fos = new FileOutputStream("output.txt");
10             FileChannel inputChannel = fis.getChannel();
11             FileChannel outputChannel = fos.getChannel()) {
12
13            ByteBuffer buffer = ByteBuffer.allocate(1024);
14            while (inputChannel.read(buffer) != -1) {
15                buffer.flip();
16                outputChannel.write(buffer);
17                buffer.clear();
18            }
19        } catch (Exception e) {
20            e.printStackTrace();
21        }
22    }
23}

```

3. **Selectors:** Allow a single thread to monitor multiple channels.

- **Example:** Using `Selector` with `SocketChannel`.

```

java
VerifyOpen In EditorRunCopy code
1import java.io.IOException;
2import java.net.InetSocketAddress;
3import java.nio.ByteBuffer;
4import java.nio.channels.Selector;
5import java.nio.channels.SelectionKey;
6import java.nio.channels.ServerSocketChannel;
7import java.nio.channels.SocketChannel;
8
9public class SelectorExample {
10    public static void main(String[] args) throws IOException {
11        Selector selector = Selector.open();
12        ServerSocketChannel serverChannel = ServerSocketChannel.open();
13        serverChannel.bind(new InetSocketAddress(8080));
14        serverChannel.configureBlocking(false);

```

```

15         serverChannel.register(selector, SelectionKey.OP_ACCEPT);
16
17         while (true) {
18             selector.select();
19             for (SelectionKey key : selector.selectedKeys()) {
20                 if (key.isAcceptable()) {
21                     SocketChannel clientChannel =
serverChannel.accept();
22                     clientChannel.configureBlocking(false);
23                     clientChannel.register(selector,
SelectionKey.OP_READ);
24                 } else if (key.isReadable()) {
25                     SocketChannel clientChannel = (SocketChannel)
key.channel();
26                     ByteBuffer buffer = ByteBuffer.allocate(256);
27                     int bytesRead = clientChannel.read(buffer);
28                     if (bytesRead == -1) {
29                         clientChannel.close();
30                     } else {
31                         buffer.flip();
32                         System.out.println("Received: " + new
String(buffer.array(), 0, bytesRead));
33                     }
34                 }
35                 selector.selectedKeys().remove(key);
36             }
37         }
38     }
39 }

```

3. Serialization and Deserialization

Serialization is the process of converting an object's state into a byte stream, while **deserialization** is the reverse process.

Key Concepts

- **Serializable Interface:** Classes must implement this interface to be serializable.
- **ObjectOutputStream:** Used for serialization.
- **ObjectInputStream:** Used for deserialization.
- **serialVersionUID:** A unique identifier for each serializable class.

Example:

```

java
VerifyOpen In EditorRunCopy code
1import java.io.*;
2
3class Demo implements Serializable {
4    public int a;
5    public String b;
6
7    public Demo(int a, String b) {
8        this.a = a;
9        this.b = b;
10    }
11}
12
13public class SerializationExample {

```



```

14     public static void main(String[] args) {
15         Demo object = new Demo(1, "example");
16         String filename = "file.ser";
17
18         // Serialization
19         try (FileOutputStream file = new FileOutputStream(filename);
20             ObjectOutputStream out = new ObjectOutputStream(file)) {
21             out.writeObject(object);
22             System.out.println("Object has been serialized");
23         } catch (IOException e) {
24             e.printStackTrace();
25         }
26
27         // Deserialization
28         Demo object1 = null;
29         try (FileInputStream file = new FileInputStream(filename);
30             ObjectInputStream in = new ObjectInputStream(file)) {
31             object1 = (Demo) in.readObject();
32             System.out.println("Object has been deserialized");
33             System.out.println("a = " + object1.a);
34             System.out.println("b = " + object1.b);
35         } catch (IOException | ClassNotFoundException e) {
36             e.printStackTrace();
37         }
38     }
39 }

```

4. Shallow Copy vs Deep Copy

4.1 Shallow Copy

- Creates a new object but copies references to the original object's fields.
- Changes to mutable fields affect both objects.

Example:

```

java
VerifyOpen In EditorRunCopy code
1class ABC {
2    int x = 30;
3}
4
5public class ShallowCopyExample {
6    public static void main(String[] args) {
7        ABC obj1 = new ABC();
8        ABC obj2 = obj1; // Shallow copy
9        obj2.x = 6;
10       System.out.println("The value of x is: " + obj1.x); // Output: 6
11    }
12}

```

4.2 Deep Copy

- Creates a new object and recursively copies all objects referenced by the original object.
- Changes to the copied object do not affect the original.

Example:

```

java

```

VerifyOpen In EditorRunCopy code

```
1class ABC {
2    int x = 30;
3}
4
5public class DeepCopyExample {
6    public static void main(String[] args) {
7        ABC obj1 = new ABC();
8        ABC obj2 = new ABC(); // Deep copy
9        obj2.x = 6;
10        System.out.println("The value of x is: " + obj1.x); // Output: 30
11    }
12}
```

Conclusion

This comprehensive overview of the `java.io` and `java.nio` packages, along with serialization, deserialization, and the concepts of shallow and deep copies, provides a solid foundation for understanding Java's input/output capabilities. Each section includes examples to illustrate the concepts effectively. If you have any further questions or need clarification on specific topics, feel free to ask!